Bank4U

Sprint 2 Technical Report

Student: Keyur | Student ID: 20231990
Course: IT7742 | Assessment 2 (Sprint 2): Customer Information Management Prototype
14 October 2026

1. Updated UML Class Diagram

The class diagram below builds directly on the Sprint 1 domain model. It adds the Customer (Model) and CustomerController (Controller) classes required for this sprint, with the Controller's full attribute list and method signatures – including parameter and return types – shown in full. The association between CustomerController and Customer shows the Controller managing the customer collection (“manages 0..*”), while the existing Customer-to-Account association (“owns 1..3”) and the Account hierarchy are unchanged from Sprint 1. The new BankingException hierarchy is shown on the right, connected to Account by a “throws” dependency, reflecting that Withdraw() on any concrete account type can now raise InsufficientFundsException or OverdraftLimitExceededException.

See the following landscape page for the full diagram.

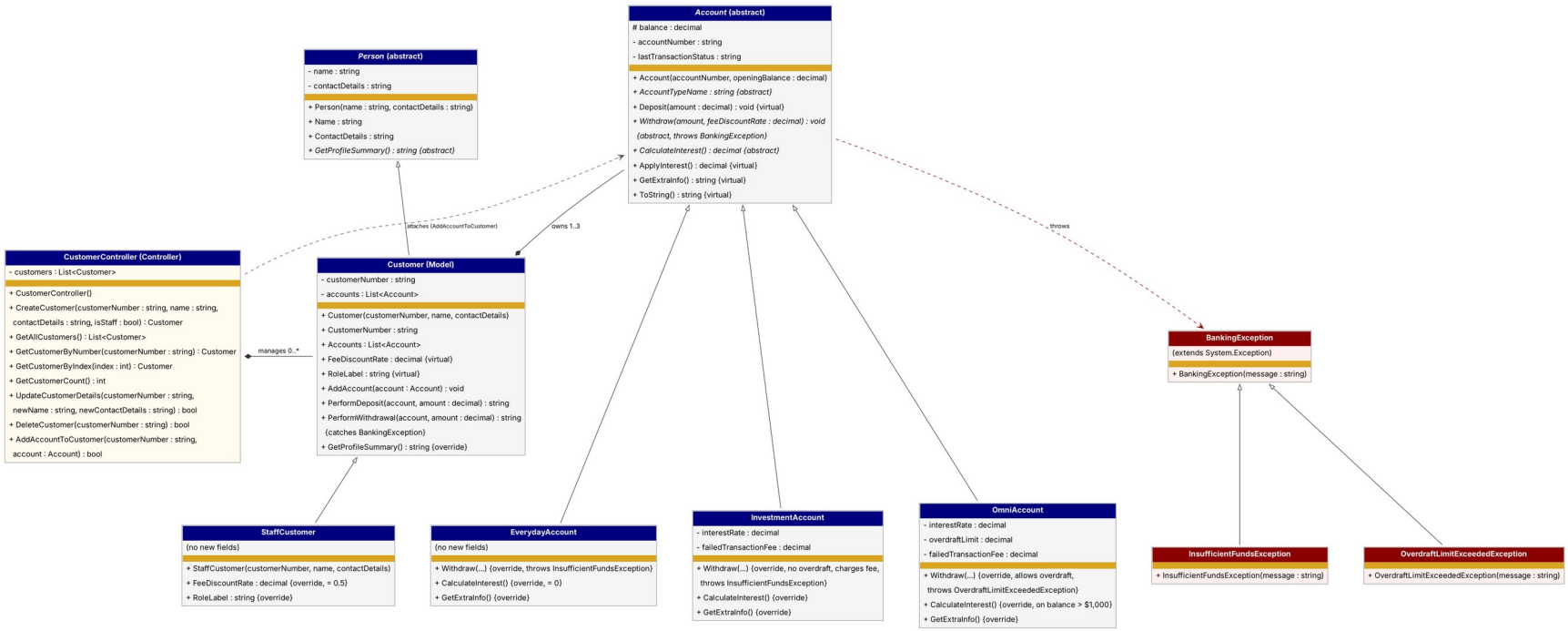


Figure 1: Updated UML Class Diagram – Customer (Model), CustomerController (Controller), the Sprint 1 Account hierarchy, and the Sprint 2 BankingException hierarchy.

2. MVC Analysis & Design

The Controller class for this sprint, CustomerController, is where every Create, Read, Update and Delete action on a customer record actually happens. CreateCustomer takes the raw details a form would collect, customer number, name, contact details, and a staff flag, and either builds a Customer or a StaffCustomer from them. Before it does that it checks whether the customer number is already taken and, if so, returns null instead of adding a duplicate, leaving the calling form to check that return value and show a message, rather than the Controller throwing an exception for what is really just an expected outcome of someone typing in an existing ID. Read is split across three methods depending on what the caller needs: GetAllCustomers for populating a list, GetCustomerByNumber when a specific record has to be found, and GetCustomerByIndex so a view that displays customers in a ListBox or ComboBox can map a selected row back to the object it represents without ever holding the list itself. Update and Delete, UpdateCustomerDetails and DeleteCustomer, both look the customer up first and return a bool so the form always knows whether the action actually happened.

The UI flexibility comes from the fact that none of this logic cares what is calling it. MainForm, CustomerForm and ManageCustomersForm all hold a reference to the same CustomerController, but not one of them stores a List<Customer> of their own. GetAllCustomers always hands back a fresh copy, so even if a form tried to add or remove something from what it gets back, the Controller's own records would be unaffected. That separation means the three Windows Forms could be replaced with a console menu, a web page, or a different GUI framework entirely, and CustomerController, Customer and the Account hierarchy underneath it would not need to change at all. The business rules live in one place, and the interface is just whatever happens to be asking it questions at the time.

3. Testing Strategy

Every withdrawal is tested along two paths: a Happy Path, where the transaction completes with no exception raised, and a Failure Path, where a business rule is broken (insufficient funds, or an exceeded overdraft limit) and Account.Withdraw throws a BankingException subclass with an account-specific message. A third case, withdrawing $0, is deliberately kept separate from both: it is an input-validation failure, not a funds/overdraft business rule, so it is still handled as a plain status message rather than an exception. Figure 2 below shows this decision logic; Table 1 lists the specific test cases – one row per automated MSTest method in the BankSprint1.Tests project – covering both paths across all three account types.

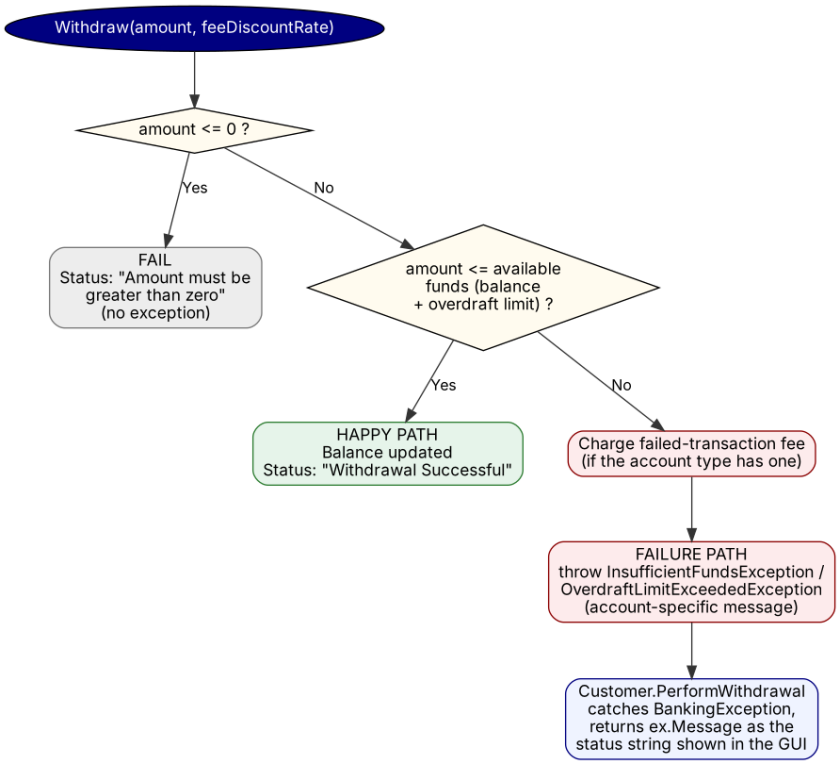


Figure 2: Withdraw() decision logic – Happy Path vs Failure Path.

Table 1: Test Case Table (Happy Path / Failure Path)

Test Case ID	Path	Input Data	Expected Result	Actual Result
TC-01	Happy	Everyday Account, balance $100, withdraw $40	Balance becomes $60; status contains “Successful”	Pass
TC-02	Happy (edge case)	Everyday Account, balance $75, withdraw $75 (exact balance)	Balance becomes $0; status contains “Successful”	Pass
TC-03	Happy (edge case)	Omni Account, balance $100, overdraft limit $200, withdraw $300 (exactly at the limit)	Balance becomes -$200; status contains “Successful”	Pass
TC-04	Happy	Omni Account, balance $1,500, rate 2%	CalculateInterest() returns $10.00 (2% of the $500 above the $1,000 threshold)	Pass
TC-05	Happy	CustomerController.CreateCustomer(“C-1”, …)	Customer created; GetCustomerCount() = 1	Pass
TC-06	Failure (validation, no exception)	Everyday Account, balance $100, withdraw $0	Balance unchanged; status contains “Amount must be greater than zero”; no exception thrown	Pass
TC-07	Failure	Everyday Account, balance $50, withdraw $50.01	InsufficientFundsException thrown; message names “Everyday Account”; balance unchanged (no fee)	Pass

Test Case ID	Path	Input Data	Expected Result	Actual Result
TC-08	Failure	Investment Account, balance $100, fee $25, withdraw $150	InsufficientFundsException thrown; balance becomes $75 (fee charged, no overdraft)	Pass
TC-09	Failure (required edge case)	Omni Account, balance $100, overdraft limit $200, fee $35, withdraw $300.01 (one cent over the limit)	OverdraftLimitExceededException thrown; balance becomes $65 (fee charged); withdrawal never applied	Pass
TC-10	Failure	Customer.PerformWithdrawal on an Everyday Account, balance $50, withdraw $100	Exception is caught internally and returned as a plain status string – does not throw out to the caller	Pass
TC-11	Failure	CustomerController.CreateCustomer called twice with the same customer number	Second call returns null; customer count stays at 1	Pass
TC-12	Failure	CustomerController.DeleteCustomer with an unknown customer number	Returns false; no exception thrown	Pass

All tests above were run and confirmed passing in Visual Studio's Test Explorer before submission.

4. GitHub Repository

Private repository (lecturer/tutor added as a collaborator): https://github.com/keyur232/20231990-bank4u-sprint1